从 PDF 到分析结论
第一性原理教学讲义

基于 Graphiti 本地部署与武汉大学舆情案例的系统推导

这不是一份“功能说明书”，而是一份解释系统为何这样设计、
每一层中间表示在解决什么问题、以及为什么最终能从 PDF 走到分析结论
的教学讲义。

案例输入文档：	武汉大学品牌声誉深度分析报告.pdf
模拟问题：	如果武汉大学发布了对肖某某处分的撤销公告，大众舆情走向会怎么样
系统形态：	Flask Backend + Graphiti Sidecar + Kuzu + OASIS 双平台模拟
运行模型：	gpt-5.4-mini
Embedding：	BAAI/bge-m3
编译方式：	XeLaTeX
日期：	2026-03-29

Codex 依据当前代码、配置与真实日志编写

摘要

这份讲义试图回答一个看似简单、实则非常容易被误解的问题：为什么一份 PDF 文档，经过一套本地系统处理之后，能够变成一份带有趋势判断、角色分化、舆情节奏和风险提示的分析报告？

如果只从表面看，这个系统像是在做“文档理解 + 大模型生成”。但从第一性原理看，它做的事情更接近于：先把原始文档压缩成可检索的世界模型，再把世界模型变成可发声的主体集合，再让主体在一个有时间和平台约束的仿真环境中演化，最后让报告智能体回到这个世界模型和仿真世界里进行二阶推理。

因此，这套系统的本质并不是“把 PDF 喂给模型，让模型直接写答案”，而是分三层连续构造：

1. 构造**可计算文本**；
2. 构造**可检索、可复用、可被推演的中间世界模型**；
3. 构造**基于世界模型的多主体演化与分析生成**。

本讲义以武汉大学案例作为真实教学样本，直接使用本次运行中的正式参数和正式结果：

- 原始 PDF 文件大小约 2.89MB，抽取后文本长度为 20862 字符；
- chunk 参数为 6500 / 650，只切成 4 个 chunk；
- 图谱规模为 64 个节点、80 条边；
- 从图谱中筛出并生成了 59 个 Agent 配置；
- 双平台仿真共执行 216 个动作；
- 报告阶段本地搜索调用 21 次，其中 18 次命中为 0；
- 全链路从项目创建到最终报告完成约 939.3 秒，即 15 分 39 秒。

这些数据非常适合作为教学材料，因为它们同时展示了系统为什么能工作，也展示了它为什么会慢、为什么会失真，以及为什么参数调优永远是在速度、召回和表达力之间做权衡。

目录

第一章　问题定义：这到底是不是一个摘要问题 **1**
1.1 表面任务与真实任务 1
1.2 为什么不能直接把 PDF 扔给大模型 1

第二章　第一性原理：为什么必须有中间世界模型 **2**
2.1 信息处理的三层目标 2
2.2 从载体到世界模型 2
2.3 每一层都在保留什么、丢弃什么、获得什么 2

第三章　系统全链路总览 **4**
3.1 真实案例的系统结构 4
3.2 从代码模块看职责分工 4

第四章　第一步：PDF 如何变成可计算文本 **6**
4.1 PDF 只是阅读容器，不是知识对象 6
4.2 本项目如何抽取文本 6
4.3 为什么文本清洗不能太激进 6

第五章　第二步：为什么必须分块 **8**
5.1 分块不是技术细节，而是系统的时间复杂度开关 8
5.2 为什么 overlap 必须存在 8
5.3 本次案例为什么只切成了 4 个 chunk 8

第六章　第三步：本体生成，本质上在做什么 **10**
6.1 本体不是标签集合，而是系统的观察框架 10
6.2 为什么这里必须先有问题，再有本体 10
6.3 本项目的本体生成约束 10
6.4 本次案例实际生成了什么 10

第七章　第四步：图谱构建，本质上在做什么 **12**
7.1 图谱是世界模型，不是数据库截图 12
7.2 Graphiti Sidecar 在做什么 12
7.3 为什么图谱是比 chunk 更强的中间表示 12
7.4 本次案例图谱结果意味着什么 13
7.5 为什么建图速度会慢 13

第八章　第五步：为什么图谱还不够，必须继续生成 Agent **14**
8.1 图谱里的实体不等于可发声主体 14
8.2 本项目如何从图谱生成 Agent 配置 14
8.3 为什么要把节点人格化 14
8.4 本次案例里的真实结果 14

第九章　第六步：社会仿真如何把结构化记忆变成动态舆情 **16**
9.1 什么叫“在图谱上做仿真” 16
9.2 本次案例的仿真配置 16

9.3 为什么仿真不是“一次多轮问答” . . . 16
9.4 这次仿真为什么看起来并不快 . . . 16
9.5 为什么仿真进程结束后还在 . . . 17

第十章 第七步：报告为什么不是直接摘要，而是二阶推理 18
10.1 报告层真正依赖的不是原文，而是两个世界 . . . 18
10.2 ReportAgent 的工作方式 . . . 18
10.3 为什么报告阶段的搜索质量如此重要 . . . 18
10.4 这次报告最后输出了什么 . . . 19

第十一章 武汉大学案例的真实运行复盘 20
11.1 时间分布 . . . 20
11.2 规模分布 . . . 21
11.3 为什么全链路比“官方演示”慢很多 . . . 21

第十二章 速度、质量与瓶颈：为什么会慢，为什么会失真 22
12.1 真正的主瓶颈：远端大模型链路 . . . 22
12.2 第二瓶颈：主体数与采访数 . . . 22
12.3 第三瓶颈：大 chunk 带来的检索退化 . . . 22
12.4 本次运行有没有错误 . . . 22

第十三章 如何做参数调优：不是越快越好，而是找到系统平衡点 24
13.1 调优要先决定优化目标 . . . 24
13.2 对本案例最有价值的调优旋钮 . . . 24
13.3 推荐的下一步实验 . . . 24

第十四章 结语：什么叫“可被推演的中间世界模型” 26

附录：本次案例的真实配置与日志摘要 27
.1 关键配置摘要 . . . 27
.2 关键时间戳摘要 . . . 27
.3 检索质量摘要 . . . 27
.4 典型日志片段 . . . 27

附录：把这份讲义复用到别的案例时，应该先改什么 29

第一章　问题定义：这到底是不是一个摘要问题

1.1　表面任务与真实任务

很多人第一次看到这个案例时，会把问题理解成："给定一份 PDF，再给定一个问题，让大模型输出一份分析报告。"这当然不是完全错，但它省略了最关键的部分：**报告中的许多内容并不直接存在于原文里，而是来自对原文结构的重建、对主体关系的外推、对多角色反应的模拟，以及对这些模拟结果的归纳。**

换句话说，系统面对的不是一个简单的"抽取事实"任务，而是一个混合任务：

- 一部分是**事实提取**：原文里提到了哪些主体、关系、事件、平台和观点；
- 一部分是**结构重建**：这些事实如何被组织成一个可检索的网络；
- 一部分是**行为建模**：如果这些主体都能发声，他们会如何响应一个新的事件假设；
- 一部分是**分析写作**：如何把上面三部分重新组织成对人可读的结论。

关键定义：什么叫"从 PDF 到分析"

所谓"从 PDF 到分析"，不是把一个文件变成一段更短的文字，而是把一个非结构化信息载体，逐层转译为可被检索、可被推演、可被审计的中间表示，最后再转译为面向人的解释性输出。

1.2　为什么不能直接把 PDF 扔给大模型

从第一性原理看，直接把 PDF 扔给大模型有四个硬伤。

第一，输入太原始。　PDF 是阅读格式，不是知识格式。它混合了版式、页面、段落、脚注和跨页断裂，对人友好，但对后续计算并不友好。

第二，信息不可复用。　如果每次都直接把整份 PDF 和问题丢给模型，那么每一次回答都是一次重新消费全部上下文，成本高，而且中间结果不可复用。

第三，信息不可检索。　系统如果想回答"哪些角色会支持""哪个平台先引爆""谁会把话题带向高校治理"，就需要一个能按实体、关系、边事实来索引的信息结构。原始 PDF 做不到这一点。

第四，信息不可推演。　"如果武汉大学发布撤销公告"是一个反事实问题。原始 PDF 提供的是历史描述，而不是主体层面的演化机制。要回答反事实问题，必须先构造可发声的主体集合和它们之间的影响网络。

本章核心结论

这套系统解决的不是"摘要"问题，而是"如何把非结构化文档转换成一个可被分析、可被搜索、可被模拟的中间世界模型"问题。报告只是这个中间世界模型的一个最终输出视图。

第二章　第一性原理：为什么必须有中间世界模型

2.1　信息处理的三层目标

如果把这类系统抽象成信息加工管线，那么它至少要满足三个目标：

1. **压缩**：把冗长文本压成较短但仍保留结构的信息表示；
2. **索引**：让系统能够按主题、实体、关系和问题进行局部访问，而不是每次全文重读；
3. **推演**：让系统能够不只回答“原文说了什么”，还回答“在这个世界里，若新增一个事件，会发生什么”。

这三个目标共同决定了为什么中间表示不能省略。

2.2　从载体到世界模型

我们可以把系统的中间表示理解成一层层“再表达”：

$$D \xrightarrow{P} T \xrightarrow{C} \{c_i\} \xrightarrow{O} \mathcal{O} \xrightarrow{G} \mathcal{G} \xrightarrow{A} \mathcal{A} \xrightarrow{\text{sim}} \mathcal{S} \xrightarrow{R} \text{Report}$$

其中：

- D 表示原始 PDF；
- T 表示被抽取并清洗后的纯文本；
- $\{c_i\}$ 表示文本块集合；
- $\mathcal{O}$ 表示本体，即“系统允许哪些主体和关系存在”；
- $\mathcal{G}$ 表示图谱，即中间世界模型；
- $\mathcal{A}$ 表示 Agent 集合；
- $\mathcal{S}$ 表示社会仿真轨迹；
- Report 是面向人的最终叙事。

中间世界模型

在本讲义中，“中间世界模型”特指图谱 $\mathcal{G}$ 及其可派生的主体集合 $\mathcal{A}$。它不是全文复制，也不是最终结论，而是介于两者之间、能够支持检索和推演的结构层。

2.3　每一层都在保留什么、丢弃什么、获得什么

层级	保留的东西	丢弃的东西	新获得的能力
PDF → 文本	原始语义内容	页面布局、视觉样式	可被程序读取
文本 → chunk	局部语义片段	长距离连续性的一部分	可分批处理、可局部写入
chunk → 本体	角色类别、关系类别	无关细枝末节	约束系统“该提取什么”

本体 + chunk → 图谱	实体、关系、事实	原文冗余表达	可检索、可索引、可统计
图谱 → Agent	可发声主体及其属性	与主体无关的孤立结构	可做行为模拟
Agent + 时间 → 仿真轨迹	平台行为、观点演化	静态视角	可回答反事实问题
图谱 + 仿真 → 报告	事实、视角、趋势判断	底层过程细节	面向人的解释与决策支持

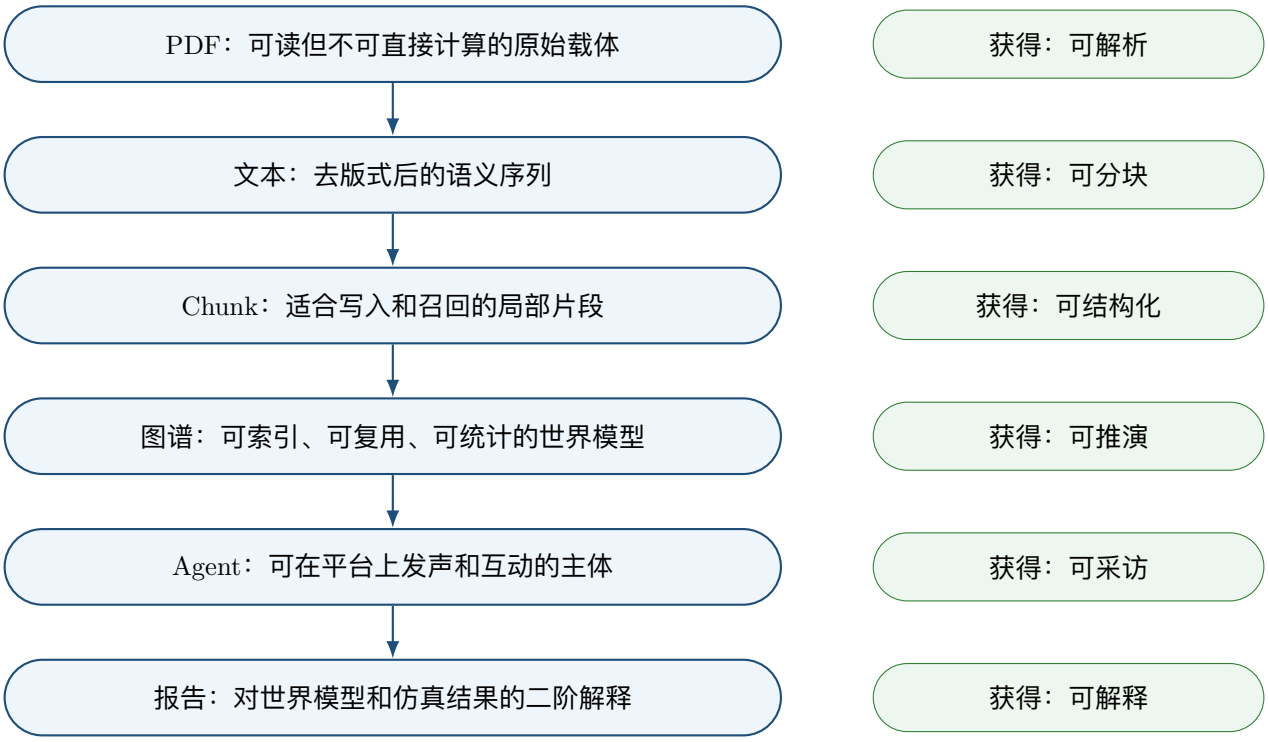

图 2.1: 信息在系统中不是简单流动，而是在每一层发生“压缩 + 重表达”

本章核心结论

如果没有中间世界模型，系统就只能反复重读全文，无法稳定检索、无法做多主体仿真，也无法把报告生成建立在可审计的结构层之上。

第三章　系统全链路总览

3.1　真实案例的系统结构

这次案例运行使用的是一套本地化但并非全离线的结构：

- 后端：Flask；
- 图谱存储与抽取：本地 Graphiti Sidecar + Kuzu；
- Embedding：本地 `BAAI/bge-m3`；
- 生成式大模型：远端 OpenAI-compatible 接口，模型为 `gpt-5.4-mini`；
- 仿真环境：OASIS 双平台并行模拟，平台为 Twitter 和 Reddit；
- 报告生成：ReportAgent，采用 ReAct 工具调用范式。

本次案例正式配置

配置项	实际值
输入 PDF	武汉大学品牌声誉深度分析报告.pdf
模拟问题	如果武汉大学发布了对肖某某处分的撤销公告，大众舆情走向会怎么样
图谱后端	graphiti
Graphiti 服务地址	`http://127.0.0.1:8011`
运行模型	`gpt-5.4-mini`
Embedding 模型	`BAAI/bge-m3`
默认 chunk size	6500
默认 overlap	650
Graph build batch size	2

3.2 从代码模块看职责分工

模块	主要职责
backend/app/utils/file_parser.py	从 PDF / MD / TXT 提取文本，并做 chunk 切分
backend/app/services/text_processor.py	统一文本预处理、分块和统计
backend/app/services/ontology_generator.py	基于模拟问题与文档内容生成实体/关系本体
backend/app/services/graph_builder.py	创建图谱、设置本体、分批写入 episode、等待构图完成
graphiti_service/service.py	本地图谱侧车服务，负责 LLM 抽取、embedding、Kuzu 存储
backend/app/services/zep_entity_reader.py	从图谱读取节点和边，过滤出可用主体
backend/app/services/simulation_config_generator.py	生成时间配置、事件配置、Agent 配置和平台配置
backend/scripts/run_parallel_simulation.py	运行双平台并行仿真，并保持 interview 命令模式
backend/app/services/report_agent.py	按章节执行 ReACT 式报告生成
backend/app/services/zep_tools.py	搜索图谱、采访 Agent、提供报告工具接口

表 3.1: 从代码角度看本系统的主路径

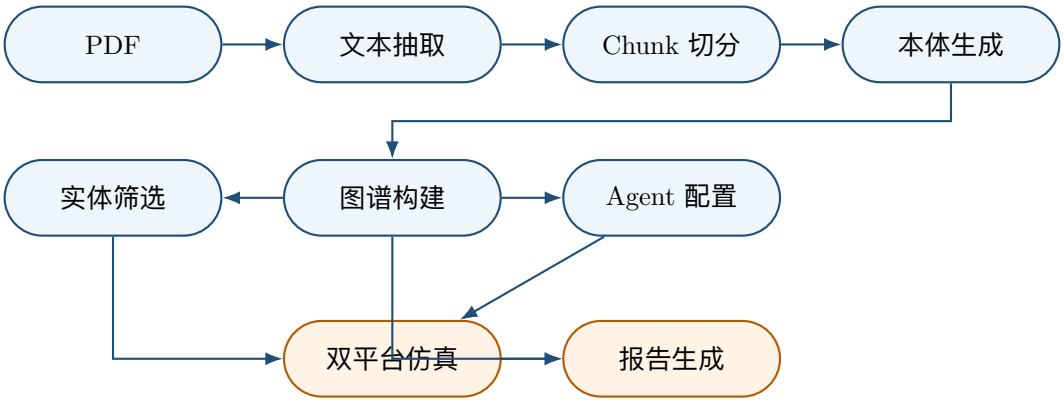

图 3.1: 本案例的真实系统主路径

本章核心结论

系统不是“一次调用一个大模型”，而是多个子系统串联：文档解析、结构化抽取、图谱持久化、主体配置、行为仿真、报告生成。每一层都在给下一层提供更高密度、更可操作的中间表示。

第四章　第一步：PDF 如何变成可计算文本

4.1　PDF 只是阅读容器，不是知识对象

从工程角度看，PDF 最难的地方不在“读不到”，而在“读到了也未必是你想要的语义顺序”。一个面向阅读的页面结构，在程序里经常表现为：

- 跨页句子被切开；
- 标题、页眉、页脚与正文混在一起；
- 原本属于同一个逻辑段落的内容在提取后变成多行；
- 表格、图片说明、引用框可能失去布局关系。

因此，第一步绝不是“拿到字符串”那么简单，而是把阅读载体转换成后续处理能够接受的语义序列。

4.2　本项目如何抽取文本

本项目的 `FileParser` 对 PDF 使用 `PyMuPDF` 逐页读取文本，然后把每页有内容的文本拼接起来。抽取完成后，`TextProcessor.preprocess_text` 会做三件很基础但必要的事情：

1. 统一换行符；
2. 压缩连续空行；
3. 去掉每一行首尾空白。

这一步看起来朴素，但它实际上在做一次**语义噪声削减**：它不理解文本含义，但它先把会干扰后续分块和提示词的格式噪声尽量抹平。

本次案例中的真实输入

- 文件：武汉大学品牌声誉深度分析报告.pdf
- 文件大小：2893754 字节，约 2.89MB
- 抽取后的总文本长度：20862 字符

4.3　为什么文本清洗不能太激进

清洗过轻，噪声残留；清洗过重，原文结构损失。比如：

- 如果把所有换行都变成空格，原本的章节边界会被抹平；
- 如果做太强的句法重写，原文措辞会被篡改；
- 如果过滤“重复句”，可能会误删报告中用于强调的关键判断。

这说明第一步的目标不是“生成更优美的文本”，而是“生成一个尽量忠实、但更适合后续分块和抽取的工作文本”。

本章核心结论

PDF 到文本的过程，是把“适合人阅读的视觉对象”转成“适合系统分块和抽取的语义序列”。这一步如果做不好，后面所有层都会被污染。

第五章　第二步：为什么必须分块

5.1　分块不是技术细节，而是系统的时间复杂度开关

一旦文本长度上来，后续所有操作都不再适合在整篇文本上一次完成。分块的核心作用是把一个长上下文问题，变成多个局部处理问题。它既影响抽取质量，也影响速度和成本。

本项目的默认切块函数采用按字符长度切分，并尝试优先在句末符号、空行或自然边界处截断。其参数只有两个：

$$\text{chunk_size} = s, \qquad \text{overlap} = o$$

其中 s 决定每块能容纳多少上下文，o 决定前后块共享多少边界信息。

5.2　为什么 overlap 必须存在

如果完全不重叠，那么位于块边界附近的实体关系可能被拦腰截断。一个典型例子是：主体在上一块出现，关系事实在下一块出现，模型就可能无法把两者连起来。

因此 overlap 的作用是：**用少量冗余换取边界连续性**。但 overlap 太大又会带来重复抽取、重复写入和更高的处理延迟。

5.3　本次案例为什么只切成了 4 个 chunk

本案例文本长度为 20862 字符，配置是：

$$\text{chunk_size} = 6500, \qquad \text{overlap} = 650$$

结果是只切成了 4 个 chunk。这个结果有明显的好处，也有明显的问题。

好处：　块数少，意味着：

- 写入 Graphiti 的 episode 数少；
- 远端 LLM 抽取调用次数少；
- 整体建图速度更快。

问题：　块太粗，对一篇高密度的中文分析报告来说，单块内部会混入太多角色、平台和叙事层次，导致：

- 细粒度事实边界变模糊；
- 局部检索的匹配词命中更差；
- 报告阶段“按问题召回相关事实”的质量下降。

本次案例中已经出现的信号

报告阶段本地搜索共调用 21 次，其中 18 次返回 0 条相关事实。这并不意味着图谱不存在，而更像是“图谱存在，但按当前问题措辞难以召回到足够相关的局部事实”。这正是大 chunk 方案的典型副作用。

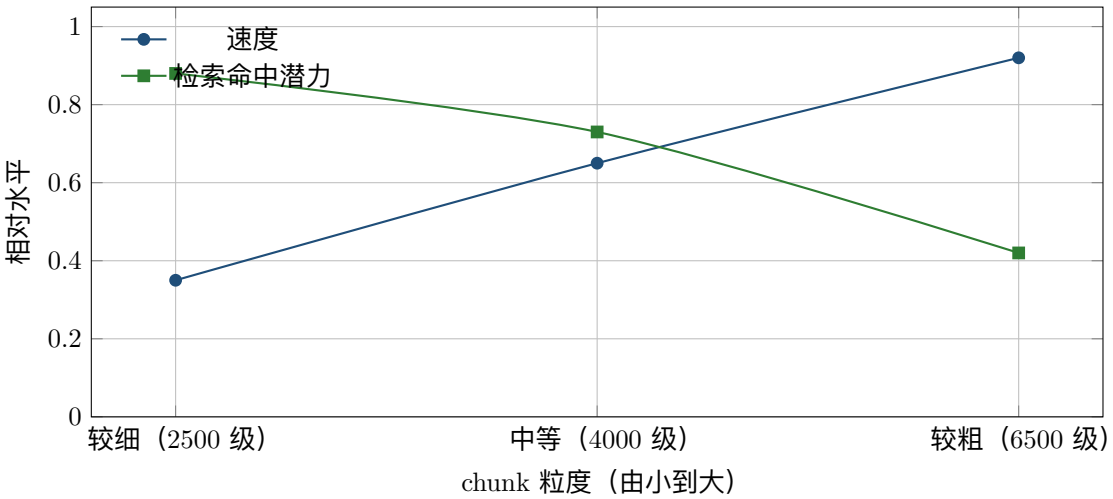

图 5.1： chunk 越大通常越快，但对密集分析型中文报告，检索命中潜力往往下降

本章核心结论

分块不是“为了适配模型上下文”这么简单，它本质上是系统在局部性、连续性和成本之间做的第一次全局权衡。

第六章　第三步：本体生成，本质上在做什么

6.1　本体不是标签集合，而是系统的观察框架

本体生成的输入是两样东西：

1. 文档文本；
2. 你想问的问题，也就是模拟需求。

这意味着本体不是从文本中“客观提取”出来的固定真相，而是**围绕任务构造的观察框架**。系统会问：如果我要模拟社交媒体舆情，那我到底需要哪些会发声的主体、哪些关系类型，以及它们之间怎样连线才有意义？

6.2　为什么这里必须先有问题，再有本体

如果问题是“总结武汉大学这份报告说了什么”，那么本体可能更偏向报告章节、指标、结论。
但我们的实际问题是：

> 如果武汉大学发布了对肖某某处分的撤销公告，大众舆情走向会怎么样？

这使得本体必须偏向“谁会说话、谁会转发、谁会对谁产生影响”的社会互动结构，而不是偏向文档结构本身。

6.3　本项目的本体生成约束

本项目的本体生成提示词给了非常强的结构约束：

- 必须正好输出 10 个实体类型；
- 最后两个必须是兜底类型：`Person` 和 `Organization`；
- 实体必须是真实世界中可以发声的主体，而不是抽象概念；
- 关系类型要服务于社交媒体舆论模拟。

这段约束极其重要，因为它回答了一个核心问题：**系统并不是什么都往图里塞，而是只允许“对后续仿真有用的主体”进入中间世界模型。**

6.4　本次案例实际生成了什么

本次案例中，本体生成阶段得到的 10 个实体类型包括：

- `University`
- `GovernmentAgency`
- `MediaOutlet`
- `SocialMediaPlatform`
- `Student`

- `Expert`
- `AlumniGroup`
- `ParentCommunity`
- `Person`
- `Organization`

从第一性原理看，这个结果说明系统已经把原始 PDF 重新看成一个“舆情主体生态”，而不是一篇普通报告。

本体层的真正作用

本体层的真正作用不是描述世界，而是规定系统将以什么粒度、什么视角、什么可操作对象去重建世界。

本章核心结论

本体生成是在回答“为了让这个问题可被模拟，我需要承认世界里有哪些会发声的对象，以及它们之间可能存在什么关系”。这是结构化建模的起点。

第七章　第四步：图谱构建，本质上在做什么

7.1　图谱是世界模型，不是数据库截图

当 chunk 和本体都准备好之后，系统开始进入真正的“中间世界模型构造”阶段。
图谱构建过程可以概括为六步：

1. 创建图谱容器；
2. 设置本体；
3. 对文本做 chunk 切分；
4. 分批把 chunk 作为 episode 写入；
5. 等待 Graphiti 处理；
6. 读取节点和边统计结果。

这六步不是机械流水线，而是在做一件非常具体的事：**把文本叙述压缩成节点、边和事实，交给图结构存储。**

7.2　Graphiti Sidecar 在做什么

本案例使用的是本地 Graphiti Sidecar。它有两个关键特点：

- embedding 在本地，使用 `BAAI/bge-m3`；
- 信息抽取使用远端 OpenAI-compatible 生成模型，本次为 `gpt-5.4-mini`。

因此，图谱构建实际上是“本地向量化 + 本地图存储 + 远端结构抽取”的混合过程。它不是纯云，也不是纯本地。

7.3　为什么图谱是比 chunk 更强的中间表示

chunk 只是切开的文本片段，它仍然是线性的。
图谱则把文本重排为：

- 节点：谁存在；
- 边：谁和谁有关；
- 边事实：这种关系在文本中被如何描述。

一旦进入图谱层，系统就可以做三件 chunk 难以高效完成的事：

1. 按主体做聚合；
2. 按关系做路径追踪；
3. 按问题做局部检索。

7.4 本次案例图谱结果意味着什么

本次运行最终得到：

$$4 \text{ chunks}, \quad 64 \text{ nodes}, \quad 80 \text{ edges}$$

这说明系统并没有把 20862 字符按句子级原样复制进来，而是压缩成了一个中等规模的关系网络。

本次案例中的图谱意义

64 个节点并不表示“只有 64 个事实”，而表示系统认为在这个舆情世界中，存在 64 个值得被识别和复用的主体或实体节点。80 条边则表示这些主体之间至少存在 80 条可用于解释和检索的显式关系。

7.5 为什么建图速度会慢

从代码流程看，真正耗时的不是单纯的 Kuzu 写盘，而是：

- 把 chunk 发送给 Graphiti Sidecar；
- Sidecar 使用 LLM 做结构抽取；
- 抽取结果再转成图结构落到本地。

所以建图时间不是磁盘 I/O 的函数，而更接近：

$$T_{\text{build}} \approx T_{\text{remote LLM extraction}} + T_{\text{embedding}} + T_{\text{graph write}}$$

其中第一项通常是主导项。

本章核心结论

图谱层的价值在于把线性文本变成可索引、可聚合、可局部访问的世界模型；建图层的成本则主要来自结构抽取，而不是图数据库本身。

第八章　第五步：为什么图谱还不够，必须继续生成 Agent

8.1　图谱里的实体不等于可发声主体

图谱里有节点，并不意味着这些节点已经具备仿真能力。图谱的节点更像“世界中的对象”，而仿真需要的是“世界中的参与者”。

因此系统需要继续做一步转换：从图谱中的实体节点里筛出那些能够承担社交媒体角色的主体，然后为它们补齐行为属性。

8.2　本项目如何从图谱生成 Agent 配置

系统会先读取图谱中的全部节点与边，再过滤出符合预定义实体类型的节点。随后，系统通过 `SimulationConfigGenerator` 分阶段生成：

1. 时间配置；
2. 事件配置；
3. Agent 配置；
4. 平台配置。

其中 Agent 配置不是一次性全生成，而是按批次生成。本项目的参数是：

$$\text{AGENTS_PER_BATCH} = 15$$

这说明系统已经意识到“主体人格化”本身也是一个长上下文问题，不能粗暴一次完成。

8.3　为什么要把节点人格化

因为图谱回答的是“谁和谁有什么关系”，而仿真回答的是“他们在什么平台、什么时间、以什么身份、用什么口吻、带着什么倾向发声”。

这两个问题不是一回事。

从实体到 Agent 的本质

实体是结构化世界里的对象；Agent 是结构化世界里的行为载体。前者解决“世界里有什么”，后者解决“这些东西会怎么动”。

8.4　本次案例里的真实结果

本次案例从图谱中生成了 59 个 Agent 配置。状态文件中的推理摘要还明确写出，这 59 个 Agent 并不是机械复制，而是经过以下几步形成：

- 根据事件性质生成 4 天的时间仿真配置；
- 根据主题设计热点话题与事件节奏；

- 生成 59 个 Agent 配置；
- 为 8 个初始帖子分配发布者。

这说明系统已经把“文档中的知识”转换成了“平台上的主体生态”。

本章核心结论

图谱是静态世界模型，Agent 集合是动态行为世界的入口。只有完成这一步，系统才能真正回答反事实舆情问题。

第九章　第六步：社会仿真如何把结构化记忆变成动态舆情

9.1　什么叫“在图谱上做仿真”

这一步最容易被误解。系统并不是在图谱上“跑图算法”，而是在一个具有平台规则、时间推进和多主体互动的仿真环境里，让 Agent 依据自己的配置和记忆进行发言、评论、点赞、转发或沉默。

于是，原来静态的结构化记忆开始转化为动态轨迹：

$$\mathcal{G} + \mathcal{A} + \tau \longrightarrow \mathcal{S}$$

其中 τ 表示时间推进机制。

9.2　本次案例的仿真配置

这次案例使用的是双平台并行仿真：

- 平台：Twitter + Reddit；
- 总模拟时长设定：96 小时；
- 但因速度约束，实际执行轮数被截断为 12 轮；
- 共 59 个 Agent 参与；
- 最终动作总数为 216，其中 Twitter 108 个，Reddit 108 个。

9.3　为什么仿真不是“一次多轮问答”

因为平台会引入新的约束：

- 不是所有人每轮都发言；
- 发言有时间分布；
- 不同平台的传播机制不同；
- 初始帖子会对后续互动路径产生偏置；
- 沉默也是一种行为。

因此仿真比普通多轮问答更接近“在规则约束下的多主体轨迹生成”。

9.4　这次仿真为什么看起来并不快

真实日志中有一个非常值得教学使用的细节：Twitter 在 16:44:05 就发出了 8 条初始帖子，而 Reddit 直到 16:47:17 才发出 8 条初始帖子。这说明双平台虽然逻辑上并行，但在真实执行中并没有完全并满。

这告诉我们一个重要事实：**系统标称并行，并不等于性能上就真正并行。**如果某一侧在等待远端模型、等待资源或等待上下文准备，整体体感仍会表现为“卡顿后跳跃”。

本次仿真的真实耗时

- Twitter 模拟循环：226.2 秒
- Reddit 模拟循环：41.5 秒
- 仿真总循环耗时：241.1 秒
- 从 API 时间戳计算的仿真区间：246.5 秒

9.5 为什么仿真进程结束后还在

这也是一个真实案例中的常见误判点。仿真脚本默认不会在模拟循环完成后立即退出，而是进入“等待命令模式”，以便后续接受 interview 命令。也就是说，**进程还活着不代表跑挂了，可能只是为了支持报告阶段继续采访 Agent。**

教学提醒

很多人看到 Python 进程不退出，会误以为系统卡住。这个案例恰好说明：应先看状态接口和脚本设计，再判断是否异常。这里进程保活是设计行为，而不是故障。

本章核心结论

仿真层把静态图谱变成动态舆情轨迹，它不是“多问几次模型”，而是在一个带平台、时间和主体差异的规则世界中生成行为序列。

第十章　第七步：报告为什么不是直接摘要，而是二阶推理

10.1　报告层真正依赖的不是原文，而是两个世界

到报告阶段时，系统同时握有两类资源：

1. 图谱世界：谁和谁有关、有哪些事实、有哪些节点和边；
2. 仿真世界：各类主体在给定反事实情境下如何行动、如何表达、如何分化。

因此报告不是在“再读一遍 PDF”，而是在这两个世界之上进行二阶组织。

10.2　ReportAgent 的工作方式

本项目中的 ReportAgent 采用 ReACT 模式，每个章节都可以调用工具。它主要依赖四类工具中的三类核心能力：

- `panorama_search`：从图谱中广角搜索全貌；
- `quick_search`：做局部快速检索；
- `interview_agents`：采访仿真中的 Agent；
- `insight_forge`：对上下文做更深层次的问题推进。

这意味着它的写作过程不是“先想好再输出”，而是“边搜索、边采访、边写作、边修正”。

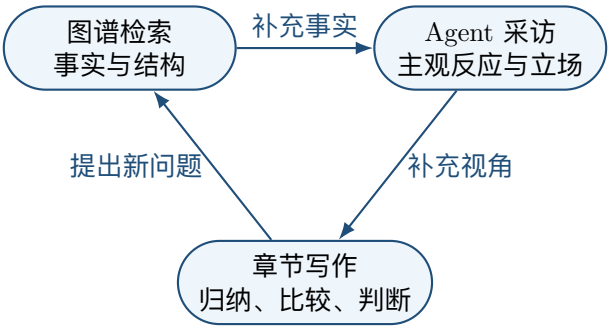

图 10.1：报告生成的本质是“检索 + 采访 + 写作”的闭环，而不是单次生成

10.3　为什么报告阶段的搜索质量如此重要

如果搜索能精准召回局部事实，报告就能建立在“结构化证据”上；如果搜索召回差，报告就会被迫更依赖 Agent 采访和模型归纳，从而：

- 事实性变弱；
- 对问题措辞更敏感；
- 更容易出现“大方向对，但局部支撑薄”的现象。

本次案例中，报告阶段的本地搜索 21 次里有 18 次命中 0 条事实，这就是一个极强的教学信号：**速度调优不能只看前半段，必须看最终写作阶段的召回代价。**

10.4 这次报告最后输出了什么

本次报告最后得出的核心判断是：撤销公告不会直接平息舆情，而更可能把讨论从“处分争议”转成“撤销争议”，并进一步外溢到程序正义、公信力、高校治理和性别议题。

这个结论之所以有教学意义，不是因为它“猜得像”，而是因为它展示了系统的生成逻辑：

1. 先从图谱层抽取已有舆情结构；
2. 再从仿真主体中获取多角色反应；
3. 最后把结构和视角拼成趋势叙事。

本章核心结论

报告层是二阶推理层。它消费的不是 PDF 原文本身，而是前面几层构造出的结构世界和行为世界。

第十一章　武汉大学案例的真实运行复盘

11.1　时间分布

本次案例的真实时间数据如下：

- 文本抽取与本体阶段：15.5 秒
- 图谱构建阶段：170.2 秒
- 仿真准备阶段：185.2 秒
- 双平台仿真阶段：246.5 秒
- 报告生成阶段：301.9 秒
- 项目创建到报告完成全链路：939.3 秒

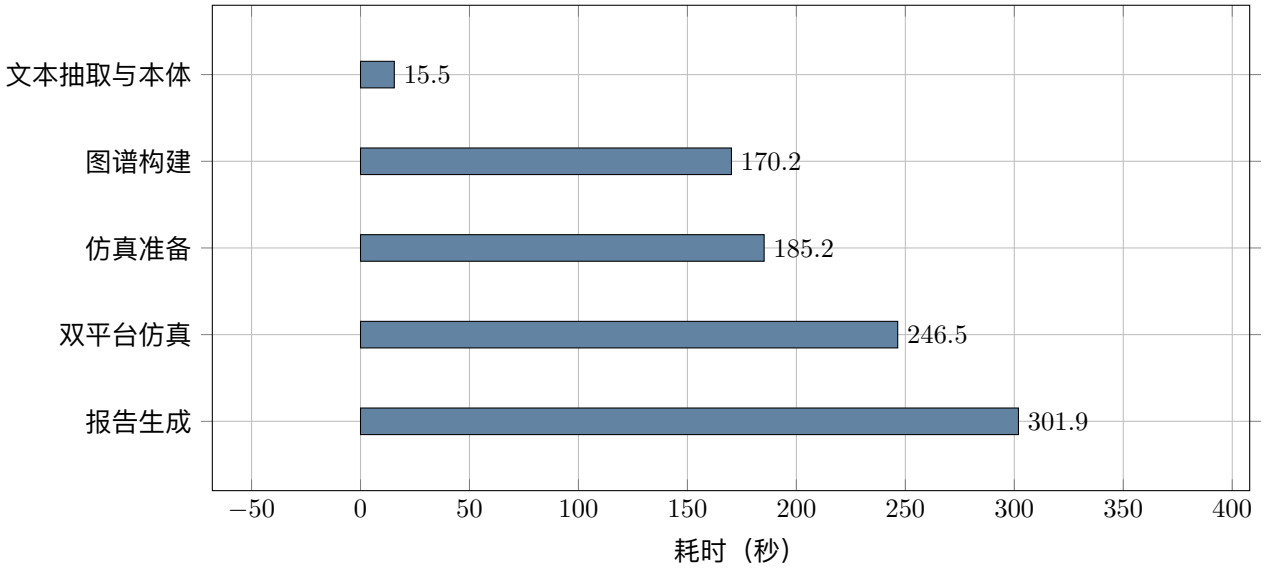

图 11.1：武汉大学案例真实运行的各阶段耗时分布

11.2 规模分布

指标	结果
原始文本长度	20862 字符
Chunk 数量	4
图谱节点数	64
图谱边数	80
可用 Agent 数	59
仿真轮数	12（由 96 小时截断而来）
总动作数	216
报告章节数	4
报告搜索调用数	21
其中 0 命中搜索	18

表 11.1：本次案例的关键规模指标

11.3 为什么全链路比“官方演示”慢很多

这是用户在真实使用中最容易提出的问题之一。对比这次案例与一些官方界面演示图，最核心的区别不是“代码逻辑有错”，而是执行环境不同：

- 本次是本地 Graphiti Sidecar + 本地 Kuzu + 远端 LLM；
- 报告阶段还要依赖真实 Agent 采访；
- 双平台仿真和报告都具有明显的多次远端模型调用。

因此，体感上的“卡顿”主要来自生成式推理链路，而不是本机 CPU 算不动。

本章核心结论

当你把真实时间分布拉平来看，会发现本系统最贵的不是“文档解析”，而是“生成配置、运行仿真、产出报告”这三步。它们都与远端推理有强耦合。

第十二章　速度、质量与瓶颈：为什么会慢，为什么会失真

12.1　真正的主瓶颈：远端大模型链路

本次案例中，最有解释力的结论是：**系统的主瓶颈不是本地图数据库，也不是本地 embedding，而是远端大模型调用链路。**

理由有三：

1. 建图、仿真准备、仿真和报告这三段都大量依赖生成式调用；
2. 日志表现为“停一段，再跳一段”，更像等待网络与接口响应，而不是持续高占用计算；
3. 仿真进程存在多条指向本地代理端口的连接痕迹，说明远端访问链路本身就是系统时延的一部分。

12.2　第二瓶颈：主体数与采访数

59 个 Agent 对于教学案例已经足够丰富，但对速度并不友好。原因在于：

- 生成 59 个 Agent 配置本身就要多轮推理；
- 双平台仿真意味着行为决策空间翻倍；
- 报告阶段多次采访 Agent，会继续消耗接口时延。

从第一性原理看，Agent 数越多，世界越像真的，但时间成本也越像真的。

12.3　第三瓶颈：大 chunk 带来的检索退化

6500 / 650 的设置让前半段更快，但它削弱了后半段。对报告来说，检索命中差的代价是：

- 需要反复遍历全图；
- 很多问题找不到直接支撑事实；
- 报告只能更多依赖采访和总结。

这就是一个非常典型的系统级 trade-off：

> 前端建图越快，不代表后端写报告越快；局部召回越差，最终分析反而可能更慢。

12.4　本次运行有没有错误

如果严格按状态接口和日志判断，本次运行**没有致命错误**：

- 仿真状态返回 `error:null`；
- 报告状态返回 `error:null`；
- 图谱构建任务成功完成；

- 最终报告文件已落盘。

但存在两个非致命 warning：

- 某些 Bert 权重未初始化；
- 某个 attention 路径退回到了 manual implementation。

这两个 warning 更像依赖库和模型兼容性信号，而不是本次流程失败原因。

本章核心结论

本次案例最值得学习的，不是“有没有报错”，而是“系统为什么在没有致命错误的前提下仍然可以表现得慢，而且部分阶段质量下降”。这正是复杂 AI 系统的常态。

第十三章　如何做参数调优：不是越快越好，而是找到系统平衡点

13.1　调优要先决定优化目标

同一套系统，至少可能有三种不同的优化目标：

1. **最短总耗时**：尽快从 PDF 跑到报告；
2. **最高报告质量**：让检索、采访和写作尽量有支撑；
3. **最好成本效率**：让质量损失在可接受范围内，但每次运行更便宜。

这三个目标通常不能同时最优。

13.2　对本案例最有价值的调优旋钮

1. chunk 参数

对高密度中文分析 PDF，建议重点比较：

- 6500 / 650：前半段快，但后半段召回压力大；
- 4000 / 400：更可能在速度与召回之间取得平衡；
- 2500 / 250：召回细，但建图和抽取调用会显著增加。

2. Agent 数量

59 个 Agent 适合展示复杂舆情生态；如果目标是更快地做业务验证，可以先压到 25–40 个。

3. 报告阶段工具调用

报告生成不是越多工具越好。若搜索质量低，增加工具调用次数只会把慢放大。

4. 模型选择

本次采用 `gpt-5.4-mini` 是速度优先思路下的选择。如果换成更强模型，可以承受更大 chunk，但未必会得到更快的全链路时间。

13.3　推荐的下一步实验

如果目标是把本案例继续向“可用产品”推进，而不是仅做一次演示，那么最值得做的是同题 A/B：

1. 保持 `gpt-5.4-mini` 不变；
2. 把 chunk 从 6500 / 650 改成 4000 / 400；
3. 比较四组指标：
    - 建图耗时；

- 图谱规模；
- 报告阶段检索命中率；
- 报告完成耗时。

这个实验之所以重要，是因为它能回答一个真正系统级的问题：**前半段变快，是否真的让最终答案更快、更稳、更有证据支撑。**

本章核心结论

调优不应该只盯住单一步骤，而应围绕“最终报告质量与总耗时”做闭环评估。系统优化的单位不是函数，而是完整链路。

第十四章　结语：什么叫“可被推演的中间世界模型”

回到最初的问题：为什么系统能从一份 PDF 走到一份分析报告？

答案不是“大模型很强”，也不是“图数据库很强”，而是这套系统在中间搭了一座桥：

$$\text{原始文档} \rightarrow \text{结构化世界} \rightarrow \text{可发声主体} \rightarrow \text{动态轨迹} \rightarrow \text{可解释结论}$$

这座桥里最关键的不是哪一个单点技术，而是中间世界模型的存在。因为只有当世界被结构化、主体被人格化、轨迹被模拟化之后，系统才有条件把“过去文本”转化为“未来判断”。

最终定义

所谓“从 PDF 到分析”的本质，是构造一个足够压缩、足够结构化、足够可检索、足够可被推演的中间世界模型，并在其上完成多主体反事实分析。

这也是为什么本案例特别适合作为教学样本。它不仅展示了系统如何成功，还展示了系统在哪里慢、在哪里丢信息、在哪里需要调优。对于产品、研究和方案设计来说，这比一次漂亮但不可解释的生成结果更有价值。

附录：本次案例的真实配置与日志摘要

.1 关键配置摘要

字段	值
GRAPH_BACKEND	graphiti
GRAPHITI_SERVICE_URL	http://127.0.0.1:8011
LLM_MODEL_NAME	gpt-5.4-mini
GRAPHITI_EMBEDDING_MODEL	BAAI/bge-m3
GRAPHITI_BUILD_BATCH_SIZE	2
DEFAULT_CHUNK_SIZE	6500
DEFAULT_CHUNK_OVERLAP	650

.2 关键时间戳摘要

阶段	时间点	说明
项目创建	16:37:37	项目开始
图谱任务创建	16:37:52	开始正式建图
图谱完成	16:40:43	图谱阶段结束
仿真准备完成	16:43:52	59 个 Agent 配置生成完成
仿真启动	16:43:53	双平台并行仿真开始
仿真完成	16:47:59	216 个动作执行完毕
报告开始	16:48:14	进入报告大纲与章节生成
报告完成	16:53:16	最终报告落盘

.3 检索质量摘要

- 报告阶段搜索调用总数：21
- 0 命中次数：18
- 非 0 命中次数：3
- 平均命中：1.714 条事实
- 最大命中：20 条事实

.4 典型日志片段

仿真日志摘录

```
[16:43:57] OASIS 双平台并行模拟
[16:43:57] [通用 LLM] model=gpt-5.4-mini
[16:44:05] [Twitter] 已发布 8 条初始帖子
[16:47:17] [Reddit] 已发布 8 条初始帖子
```

```
[16:47:58] 模拟循环完成! 总耗时: 241.1 秒
```

报告日志摘录

```
[16:48:14] 图谱搜索: graph_id=mirofish_62ade01a1fa44a8e
[16:48:14] 使用本地搜索
[16:48:14] 本地搜索完成: 找到 0 条相关事实
[16:48:22] 大纲规划完成: 4 个章节
[16:53:16] 报告生成完成: report_ffd323a8b87f
```

附录：把这份讲义复用到别的案例时，应该先改什么

如果你想把这份讲义方法迁移到别的项目，优先改以下五项，而不是从头重写全部概念：

1. 输入文档类型和长度；
2. 任务问题，即反事实问题或分析问题；
3. 本体约束，即允许哪些类型的主体进入世界模型；
4. 平台与主体规模，即仿真生态的复杂度；
5. 调优目标，是偏速度、偏质量还是偏成本。

只要这五项被改清楚，整套“PDF $\rightarrow$ 世界模型 $\rightarrow$ 仿真 $\rightarrow$ 报告”的第一性原理仍然成立。